SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Engineer to Excel

COURSE NAME: **Theory of Computation** **COURSE CODE: CSA13**

COURSE OUTCOME COVERED

CO1: *Apply mathematical foundations such as formal proofs, induction, and basic concepts of automata theory to solve computational problems. (BL:3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Duration : 1 Hour

Simulation

Code of Conduct:

I _________V.Navyasree(192373050)___________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30	3	3	3	3	3	15
Accuracy of Answer	25	2.5	2.5	2.5	2.5	2.5	12.5
Application of Knowledge	20	2	2	2	2	2	10
Completeness of Response	15	1.5	1.5	1.5	1.5	1.5	7.5
Clarity & Presentation	10	1	1	1	1	1	5
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

1 Design a **Non-Deterministic Finite Automaton (NBA)** for an **Automatic Door System** based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: *Closed, Opening, Open,* and *Closing*. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the *Closed* state. When a person is detected (P), the system may non-deterministically transition from *Closed* to *Opening*. Once opened, the system moves to the *Open* state. If no person is detected (N), the door transitions to *Closing* and eventually returns to the *Closed* state. Non-determinism occurs when the system may remain in the *Open* state even if a person is detected or may start closing if no detection occurs.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

3 Design DFA using simulator to accept the string the end with ab over set {a,b)
W= aaabab

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings "c", "bc", and "bcaaa" over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string "c" should be accepted directly from the start state, while "bc" should pass through states representing "b" and then "c". Similarly, "bcaaa" should be processed through a sequence of states that track each additional "a" after "bc". Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either "a" or "b". The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either "a" or "b". After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of "a" and "b", thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

Q1. Automatic Door System using NFA

Simple Theory

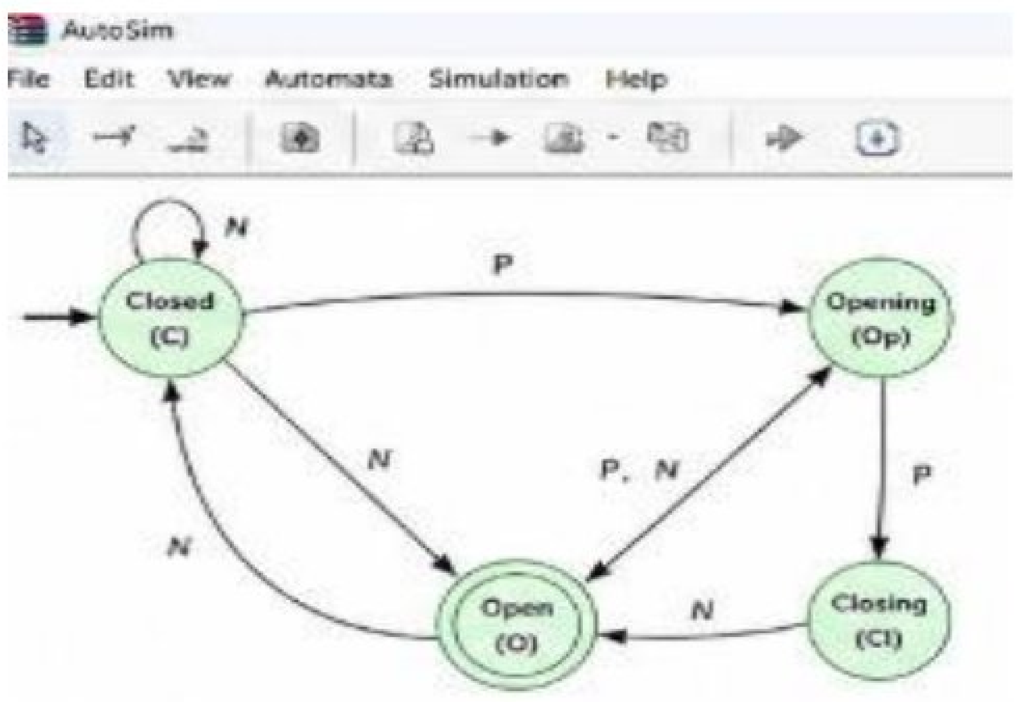
An automatic door system can be represented using an NFA with four states: Closed, Opening, Open, and Closing. The inputs are P (Person detected) and N (No person detected). The door starts in the Closed state. When a person is detected, the door starts Opening and reaches the Open state. When no person is detected, the door may move to Closing and then return to Closed. Non-determinism occurs when more than one transition is possible from a state.

Transition Table

Current State	P	N
Closed	Opening	Closed
Opening	Open	Open
Open	Open / Closing	Closing
Closing	Opening	Closed

Initial State: Closed
Final State: Open

Output:



Q2. Traffic Light Control System

Simple Theory

A traffic light system can be represented using a finite automaton with three states: Red, Green, and Yellow. The system changes from one state to the next when the timer expires. The standard sequence is Red → Green → Yellow → Red. This process repeats continuously.

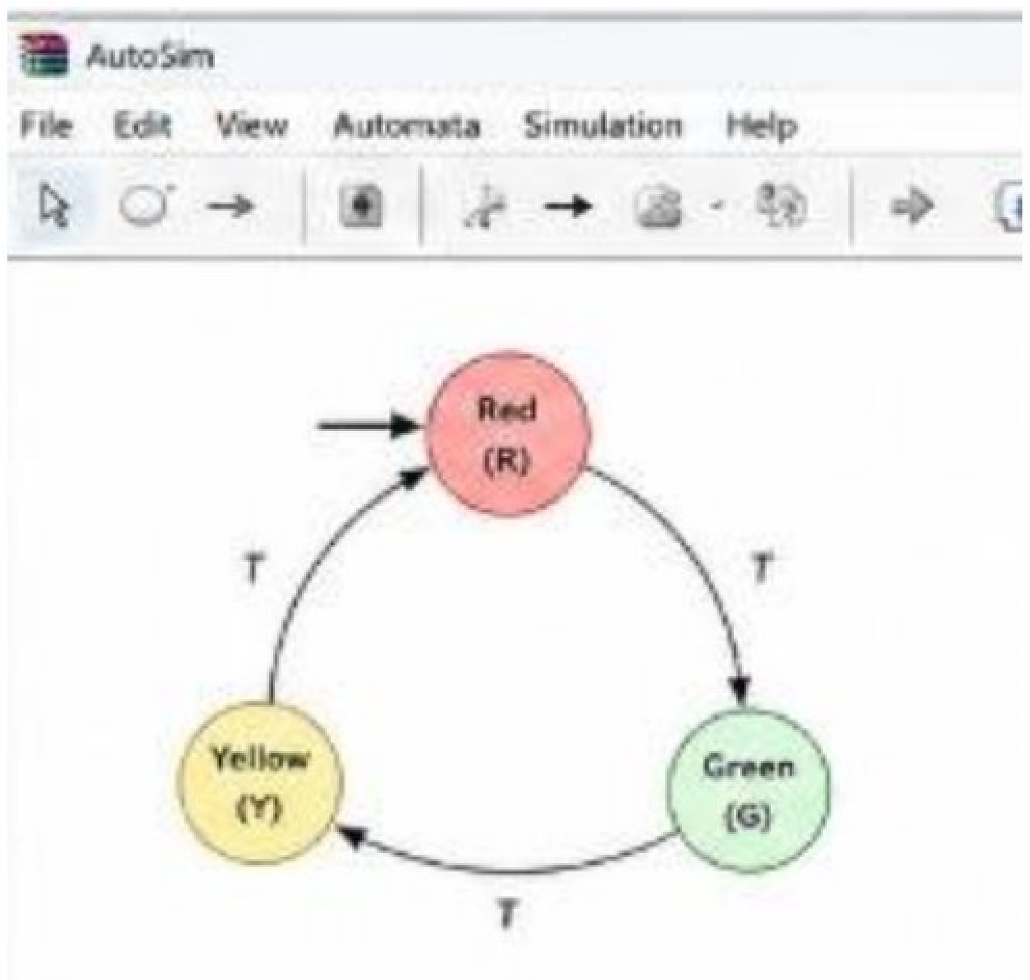
Transition Table

Current State	Input	Next State
Red	Timer	Green
Green	Timer	Yellow
Yellow	Timer	Red

Initial State: Red

The system can be extended with pedestrian crossing signals and emergency vehicle override signals.

Output:



Q3. DFA Accepting Strings Ending with ab

Simple Theory

This DFA accepts strings over the alphabet $\{a, b\}$whose last two symbols are ab. The given string is:

$$W = aaabab$$

The DFA remembers whether the last symbol is a. If the next symbol is b, it reaches the final state.

Transition Table

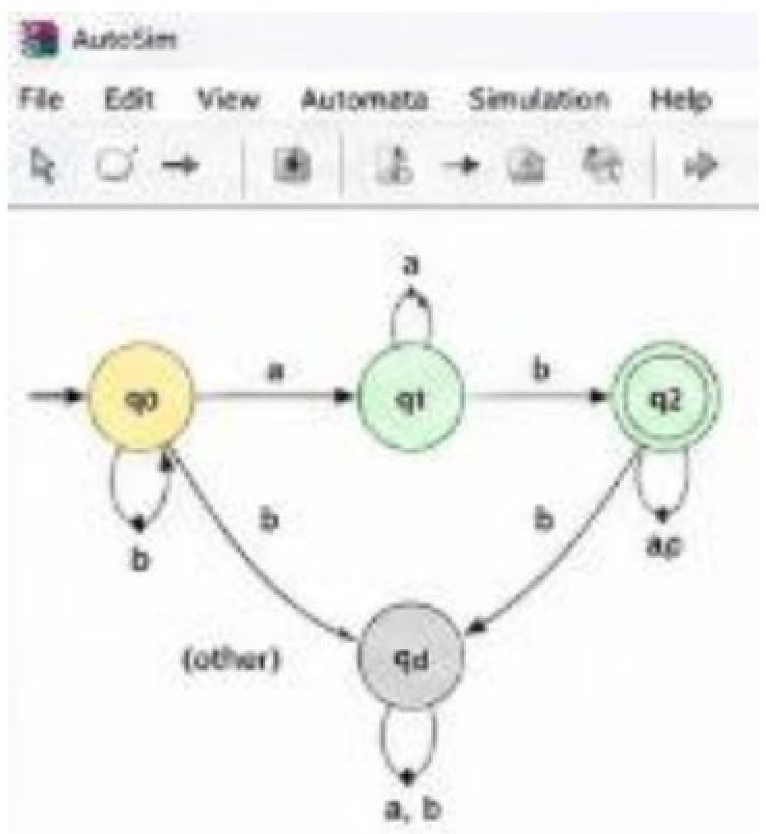
State	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_1	$\bigstar\ q_2$
$\bigstar\ q_2$	q_1	q_0

Start State: q_0
Final State: q_2

Simulation

$$q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2 \xrightarrow{a} q_1 \xrightarrow{b} q_2$$

Output:



Q4. DFA Accepting Only c, bc, and bcaaa

Simple Theory

This DFA accepts only three specific strings:

$$L = \{c, bc, bcaaa\}$$

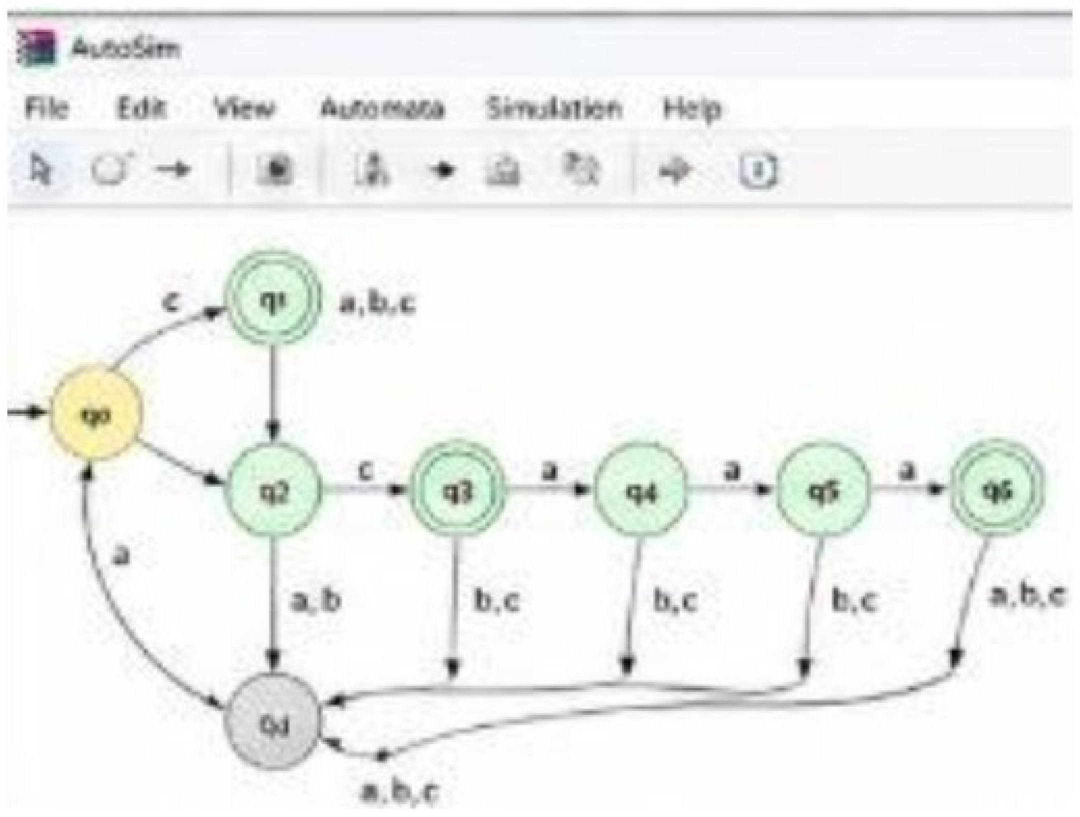
Each valid string follows a unique path from the initial state. Any other input goes to the dead state and is rejected.

Transition Table

State	a	b	c
$\rightarrow q_0$	qd	q_2	$\star\ q_1$
$\star\ q_1$	qd	qd	qd
q_2	qd	qd	$\star\ q_3$
$\star\ q_3$	q_4	qd	qd
q_4	q_5	qd	qd
q_5	$\star\ q_6$	qd	qd
$\star\ q_6$	qd	qd	qd
qd	qd	qd	qd

Start State: q_0
Final States: q_1, q_3, q_6

Output:



Q5. DFA Accepting All Non-Empty Strings over $\{a, b\}$

Simple Theory

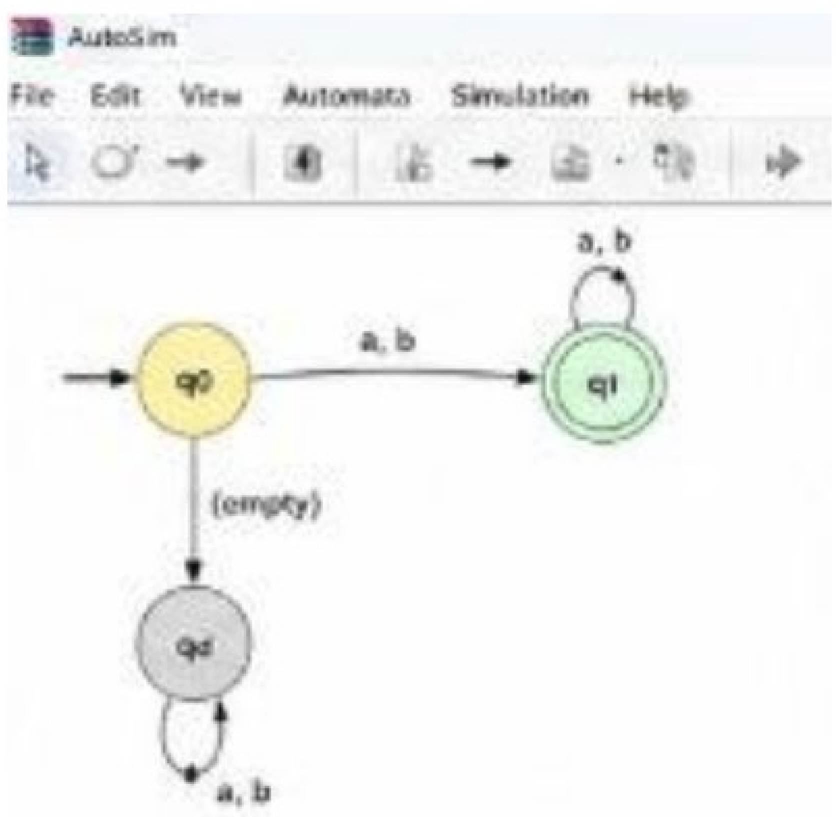
This DFA accepts all non-empty strings over the alphabet $\{a, b\}$. The first input symbol can be either a or b. After reading the first symbol, the automaton enters the accepting state and remains there for all subsequent a and b symbols.

Transition Table

State	a	b
$\rightarrow q_0$	$\bigstar\, q_1$	$\bigstar\, q_1$
$\bigstar\, q_1$	q_1	q_1

Start State: q_0
Final State: q_1

Output:



RUBRICS FOR CALCULATION

100